Modernizing RDP Certificates – Michael Waterman

michaelwaterman.nl/2026/02/08/modernizing-rdp-certificates

Michael Waterman February 8, 2026

This post is a small (promise!) but practical addition to my PKI series.

I wrote this blog because I wanted my own version of “how to set up RDP certificates.” Mostly as personal documentation, I can’t remember everything off the top of my head anymore… getting old.

During my initial research, I was surprised to see that many blogs still recommend older practices, SHA1 signatures, legacy cryptographic providers, or RSA 2048-bit keys without much explanation. And to be clear, I’m not saying RSA 2048 is bad. It’s absolutely still secure and widely used.

But RSA does have one downside, it gets exponentially slower as key sizes increase. That got me thinking, why not try Elliptic Curve instead? Is it supported for RDP? Does it actually work in practice? I couldn’t find much concrete information on the topic. Maybe my Google-Fu was failing, but it seemed like a lazy Sunday well spend researching the topic. So instead of guessing, I spent an afternoon testing, researching, and writing some PowerShell along the way, all so you don’t have to. In this post I’ll show you how to:

- Set up a modern certificate template for Remote Desktop
  - Use the Key Storage Provider (KSP) instead of the legacy CSP
  - Enforce Elliptic Curve 256 (*roughly equivalent to RSA 3072-bit*)
  - Ensure certificates are signed with SHA256
- Configure a GPO with:
  - Automatic Remote Desktop configuration
  - Automatic deployment of RDP certificates
- And most importantly: validate the result with PowerShell

Let dive in!

Build the RDP certificate template

Before I can deploy anything, I first need a proper certificate template.And since the goal of this post is to move away from older practices, I’m going to do this the modern way: Elliptic Curve cryptography only**.** Most existing guides focus on RSA-based templates. That works fine, but RSA keys become large and computationally expensive as you increase security levels. With ECC I can achieve the same security with much smaller and faster keys. For this setup I will use:

- Elliptic Curve P-256

- SHA256 signatures
- The Microsoft Key Storage Provider
- A dedicated EKU for Remote Desktop

Let's build it from scratch.

Step 1 – Verify the RDP EKU OID

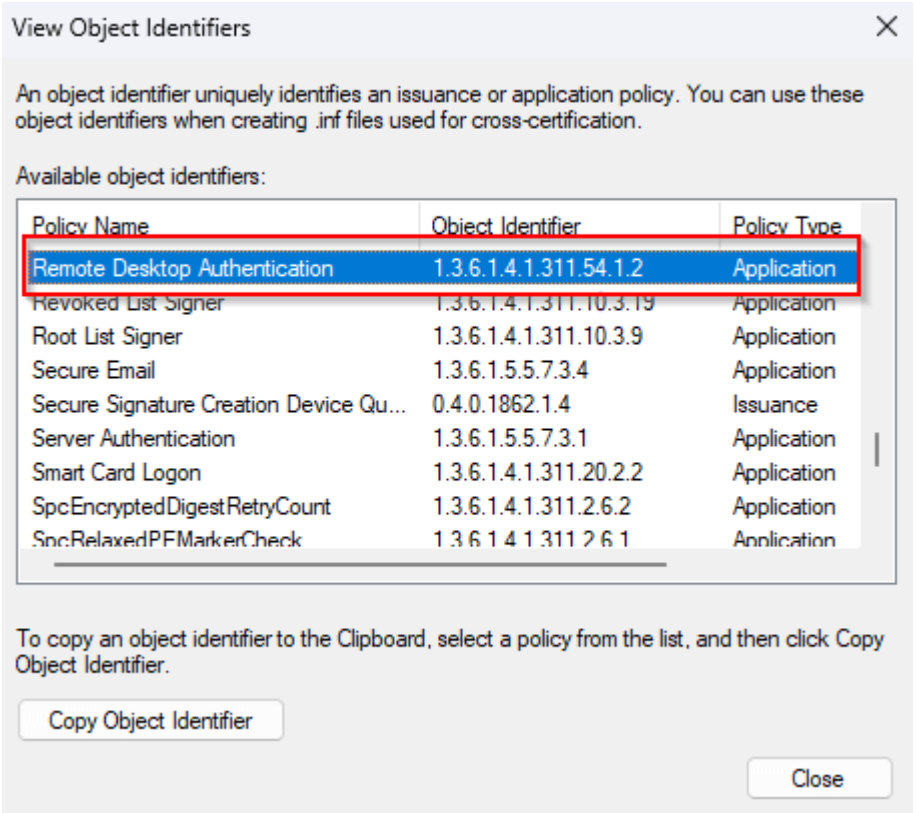
Remote Desktop certificates require a specific Enhanced Key Usage identifier:

1.3.6.1.4.1.311.54.1.2

Before creating anything, it's a good idea to check whether this OID is already available in your environment. You can verify this in the Certificate Templates console:

1. Open *"certtmpl.msc*"
2. Expand "*Object Identifiers*"
3. Look for an entry named something like "*Remote Desktop Authentication*"
4. Confirm that it has OID "*1.3.6.1.4.1.311.54.1.2*"

If it's not available, I show you how to create it in the next step as part of the template configuration.

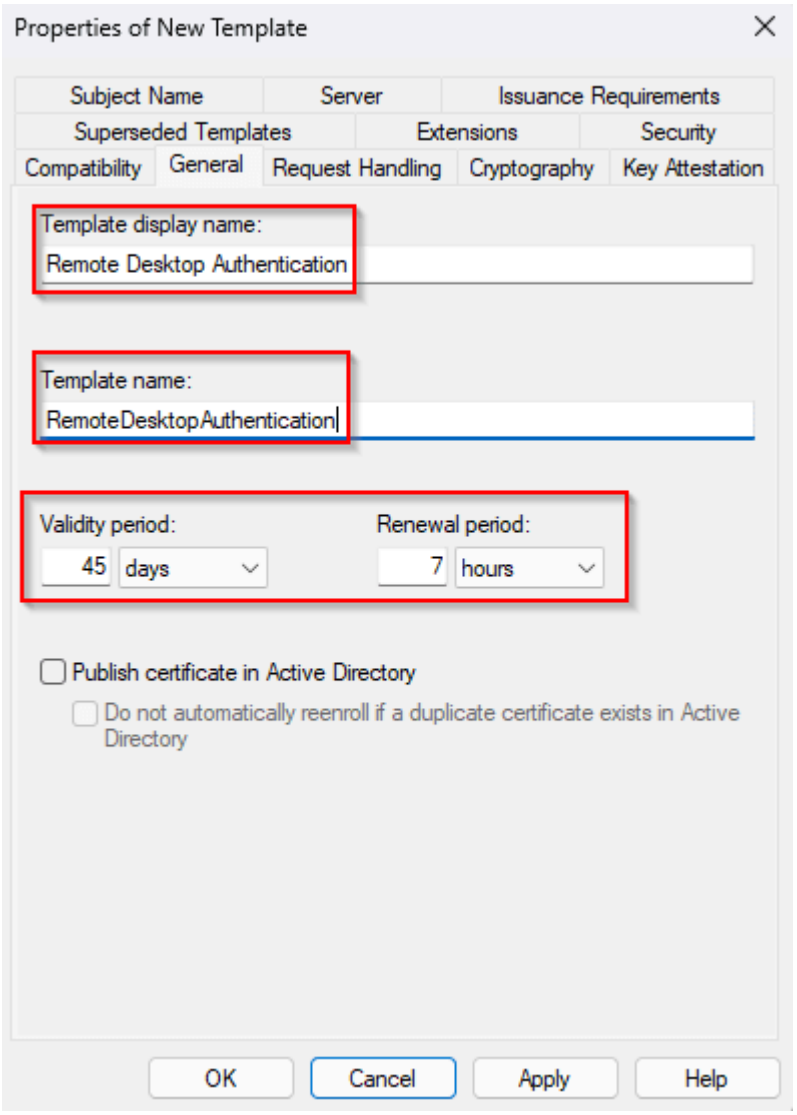


An already existing Remote Desktop OID

Step 2 – Create the template

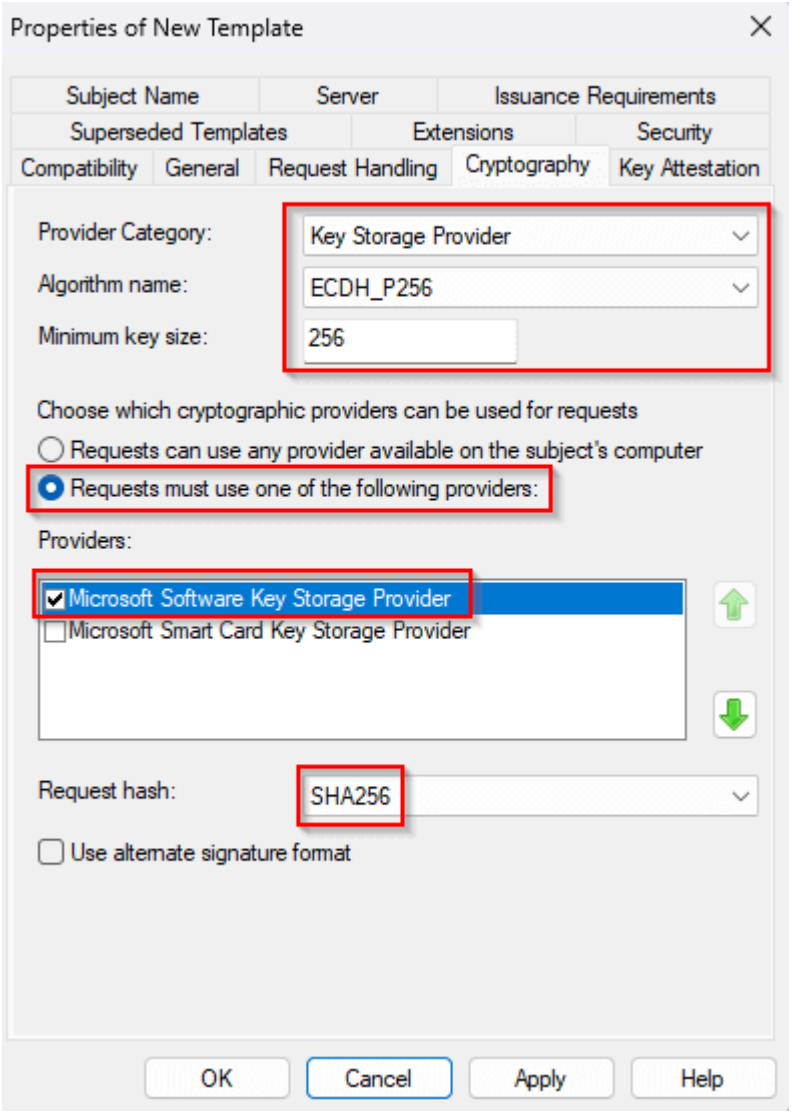
Open the Certificate Templates console and:

- Locate the built-in "*Computer*" template
- Right-click → *Duplicate Template*
- On the "*Compatibility*" tab
  - Certification Authority: *Windows Server 2016*
  - Certificate recipient: *Windows 10 / Windows Server 2016*
- On the "*General*" tab:
  - Template display name: *Remote Desktop Authentication*
  - Validity period: *45 days*, anticipating the coming changes
  - Renewal period: *7 hours*, in line with the previous remark



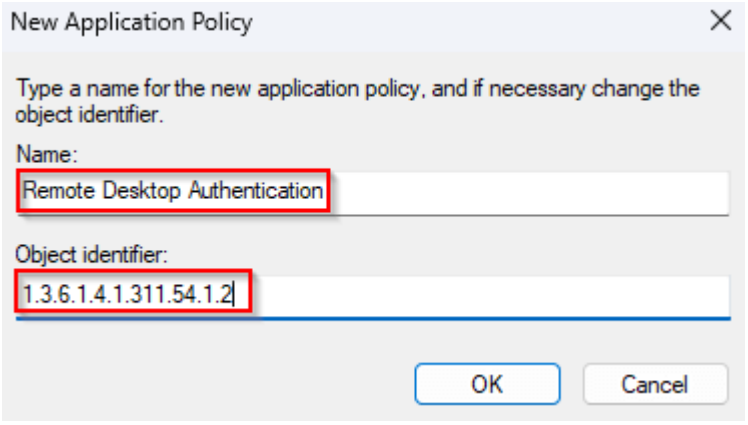
On the *"Cryptography"* tab:

- Provider Category**:** *Key Storage Provider*
- Algorithm name: *ECDH_P256*
- Minimum key size: *256*
- Provider: *Microsoft Software Key Storage Provide*r
- Request hash: *SHA256*



On the "*Extensions*" tab:

- Select the "*Application Policies*" → "Edit"
- Remove the "*Client Authentication*" & "*Server Authentication* (*Please note that Server Authentication is a requirement according the documentation found here. Testing has shown that this is only for a standalone server and has been enforced as of server 2025*)"
- Click "*Add*" → "*New*"
  - Name: *Remote Desktop* Authentication
  - Object Identifier: *1.3.6.1.4.1.311.54.1.2*
- Click "*OK*" twice.



- On the "*Subject Name*" tab:
  - Select "*Build from this Active Directory information*"
  - Subject name format: *DNS Name*
  - Include this information in subject alternate name: *DNS*
- On the "*Security*" tab:
  - Add "*Domain Controllers*", *Enroll, Autoenroll*
  - Add "Domain Computers", *Enroll, Autoenroll*
  - leave the remaining defaults
- Click "*OK*"

Step 2 -Publish the template

Finally, make the template available on the CA. On the Certification Authority server:

- Open the *Certification Authority* console
- Right-click *Certificate Templates*
- Choose *New → Certificate Template to Issue*
- Select your new *Remote Desktop Authentication* template

In the next chapter I'll configure Group Policy to automatically deploy this certificate and bind it to Remote Desktop.

Configuring the Group Policy object

In this chapter I'll configure the Group Policy settings required to deploy and activate our Remote Desktop certificates. Important note, this post assumes that *certificate auto-enrollment is already configured in your environment*. The settings below only cover the RDP-specific configuration and firewall rules.The configuration consists of three main parts:

1. Windows Firewall rules
2. Remote Desktop enabling
3. Certificate template binding

Below are the exact paths and settings as configured.

Windows Firewall – Allow RDP Traffic

```
Computer Configuration
 └ Policies
   └ Windows Settings
     └ Security Settings
       └ Windows Firewall with Advanced Security
         └ Inbound Rules
```

Add these default rule settings:

- *Remote Desktop – Shadow (TCP-In)*
- *Remote Desktop – User Mode (TCP-In)*
- *Remote Desktop – User Mode (UDP-In)*

Security configuration and certificate binding

```
Computer Configuration
 └ Policies
   └ Administrative Templates
     └ Windows Components
       └ Remote Desktop Services
         └ Remote Desktop Session Host
           └ Security
```

Configure the following settings:

- Require user authentication for remote connections by using Network Level Authentication: *“Enable*“
- Server authentication certificate template: “*RemoteDesktopAuthentication*“

Tip! You can also use the OID of the certificate template. It’s located in the “Extension” tab or use this PowerShell code (yeah I know, wayyy cooler right)

```
Get-ADObject -LDAPFilter "(&(objectClass=pKICertificateTemplate)
(cn=RemoteDesktopAuthentication))" `
        -SearchBase "CN=Certificate Templates,CN=Public Key
Services,CN=Services,CN=Configuration,DC=corp,DC=michaelwaterman,DC=nl" `
        -Properties msPKI-Cert-Template-OID
```

```
DistinguishedName       : CN=RemoteDesktopAuthentication,CN=Certificate Templates,CN=Public Key
                          Services,CN=Services,CN=Configuration,DC=corp,DC=michaelwaterman,DC=nl
msPKI-Cert-Template-OID : 1.3.6.1.4.1.311.21.8.3257360.1500923.5195963.8254110.15234830.123.2219732.1253337
Name                    : RemoteDesktopAuthentication
ObjectClass             : pKICertificateTemplate
ObjectGUID              : 1212e6e1-f898-4d59-9a63-a775b30fdd44
```

If your not a PowerShell person (why not!?) but still want to quickly verify how a template is actually stored in Active Directory, use:

```
certutil -adtemplate -v RemoteDesktopAuthentication”
```

Enable Remote Desktop

```
Computer Configuration
 └ Policies
   └ Administrative Templates
     └ Windows Components
       └ Remote Desktop Services
         └ Remote Desktop Session Host
           └ Connections
```

Configure the following settings:

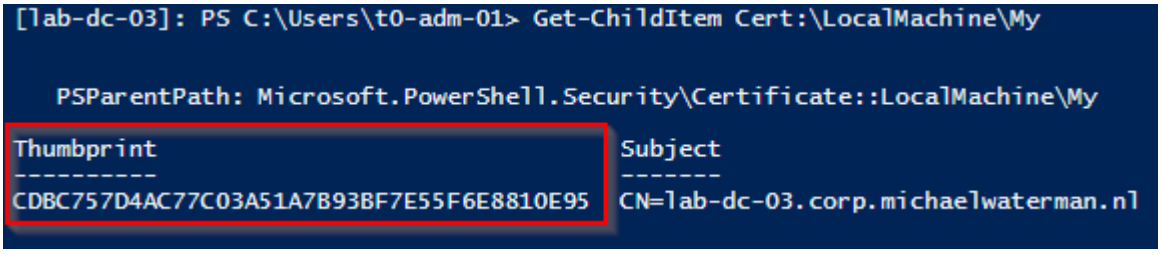
> Allow users to connect remotely by using Remote Desktop Services: “*Enable*“

With the GPO configured, all that’s left is a bit of patience. By default it can take up to 90 minutes before the new settings apply, or you can force an immediate update using:

Validation

Up to this point everything has been configuration, creating the template and setting up the GPO. Now comes the important part, making sure it actually worked. First, a small reality check. Even after the GPO has been applied, it can take a little while before the new certificate actually appears on the system. Auto-enrollment doesn’t always happen instantly. So don’t panic if nothing shows up right away. The easiest first step is simply to look in the local machine certificate store:

```
Get-ChildItem Cert:\LocalMachine\My
```



At some point you should see a new certificate appear that matches the Remote Desktop template. If you don’t want to wait, you can always force things a bit by refreshing Group Policy. After a policy refresh and a short wait, the certificate should be enrolled automatically.

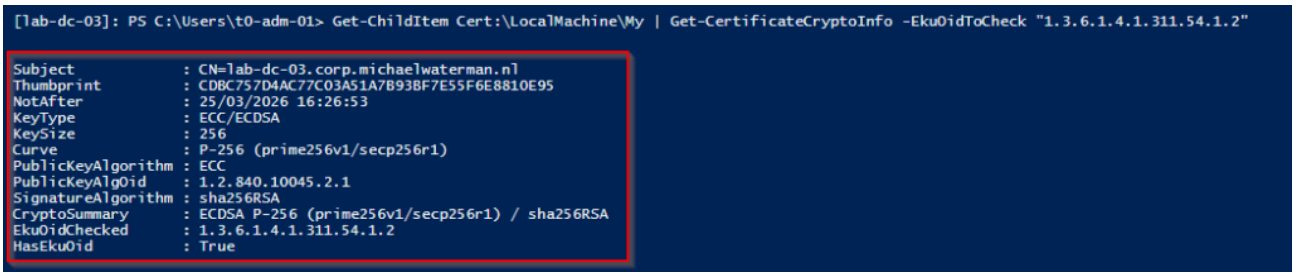
Inspect the certificate

Seeing a certificate in the store is a good start, but it doesn't tell the whole story.I want to know exactly what kind of certificate it is. That's where my `Get-CertificateCryptoInfo` function comes in (yes, again PowerShell). Using this script I can verify:

- Whether the certificate is RSA or ECC
- Which curve or key size is being used
- The signature algorithm
- And whether the correct EKU is present

Simply run:

```
Get-ChildItem Cert:\LocalMachine\My | Get-CertificateCryptoInfo -EkuOidToCheck
"1.3.6.1.4.1.311.54.1.2"
```



This gives a clear, human-readable overview of the cryptographic properties of the certificates on the system. As you can see the certificate that matches the settings from the template has the intended ECC P-256, SHA256 and the Remote Desktop OID. You can get the script from [my GitHub repository here](#).

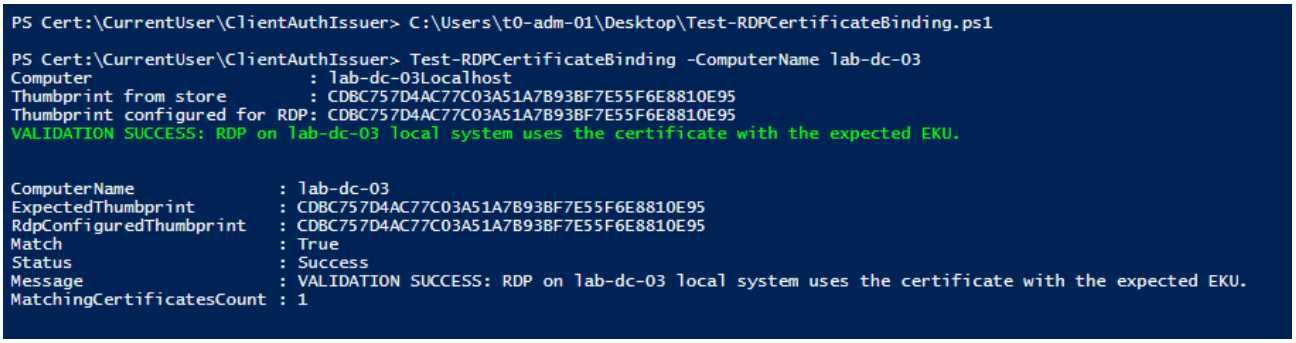
Validate from a client perspective

A certificate in the store is great, but the real question is:

> *Is Remote Desktop actually using it?*

To answer that, I'll switch to a Windows 11 client and use a client side validation function I created specifically for this purpose:

```
Test-RDPCertificateBinding -ComputerName <servername>
```



As you can see in the image above, the thumbprint matches the exact same thumbprint I generated with the `Get-CertificateCryptoInfo` function on the server itself, validating that this certificate is the one used with the RDP connection. You can get my script from `Test-RDPCertificateBinding` my GitHub repository here.

And that's it. You can now connect to your remote machines over RDP without any warnings, confident that the connection is secured with properly configured, modern encryption.

In conclusion

Well, this turned out to be a fun little project. What started as "let me quickly set up RDP certificates" turned into a deep dive into ECC, templates, and validation. The result is a setup that's modern, secure, and easy to maintain. No legacy providers, no weak algorithms, and no certificate warnings, just a solid configuration you can trust, exactly how it should be. Hope I kept my promise and gave you a small, practical and to the point blog!

As always, feedback is very welcome! Until next time…

References

Use certificates in Remote Desktop Services | Microsoft Learn (Thanks to André Stuhr, check out his website here)